

1 while 程序指称语义

定义 1.1 (指称语义)

对程序 S , 定义其指称语义

$$\llbracket S \rrbracket \stackrel{\text{def}}{=} \{(\sigma, \tau) \in \Sigma^2 \mid \langle S, \sigma \rangle \text{ 终止于 } \tau\},$$

显然 $\llbracket S \rrbracket$ 是 Σ 上的部分函数.

考虑 Σ 上的所有部分函数集 $[\Sigma \rightarrow \Sigma]$ 的包含偏序, 显然是一个 CPO.

定理 1.1 (skip 与 assign 语句的指称语义)

对任意的同类型的变量 u 和项 t , 以及状态 σ , 有

$$\llbracket \text{skip} \rrbracket(\sigma) = \sigma, \quad \llbracket u := t \rrbracket(\sigma) = \sigma[u := \sigma(t)],$$

定理 1.2 (seq 语句的指称语义)

对任意的程序 S_1, S_2 有 $\llbracket S_1; S_2 \rrbracket = \llbracket S_2 \rrbracket \circ \llbracket S_1 \rrbracket$.

证明.

任取状态 σ 和从 $\langle S_1, \sigma \rangle$ 开始的计算 $\langle S_1^i, \sigma^i \rangle$, 归纳易证 $\langle S_1^i; S_2, \sigma^i \rangle$ 是一个转移序列.

如果 $\langle S_1, \sigma \rangle$ 终止于 σ' 且 $\langle S_2, \sigma' \rangle$ 终止于 τ , 那么

$$\langle S_1, \sigma \rangle \rightarrow^* \langle E, \sigma' \rangle \implies \langle S_1; S_2, \sigma \rangle \rightarrow^* \langle E; S_2, \sigma' \rangle = \langle S_2, \sigma' \rangle \rightarrow^* \langle E, \tau \rangle,$$

此时 $\llbracket S_1; S_2 \rrbracket(\sigma) = \tau = \llbracket S_2 \rrbracket(\sigma') = \llbracket S_2 \rrbracket \llbracket S_1 \rrbracket(\sigma)$.

如果 $\langle S_1, \sigma \rangle$ 终止于 σ' 且 $\langle S_2, \sigma' \rangle$ 发散, 那么

$$\langle S_1, \sigma \rangle \rightarrow^* \langle E, \sigma' \rangle \implies \langle S_1; S_2, \sigma \rangle \rightarrow^* \langle E; S_2, \sigma' \rangle = \langle S_2, \sigma' \rangle \text{ 发散},$$

此时 $\llbracket S_1; S_2 \rrbracket(\sigma)$ 无定义, 同时 $\llbracket S_2 \rrbracket \llbracket S_1 \rrbracket(\sigma) = \llbracket S_2 \rrbracket(\sigma')$ 无定义.

如果 $\langle S_1, \sigma \rangle$ 发散, 那么 $\langle S_1; S_2, \sigma \rangle$ 也发散, 此时两边也都无定义. □

定理 1.3 (if 语句的指称语义)

对任意的 $B : \text{Bool}$ 和程序 S_1, S_2 , 有

$$\llbracket \text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi} \rrbracket = \llbracket S_1 \rrbracket \upharpoonright_B \cup \llbracket S_2 \rrbracket \upharpoonright_{\neg B}.$$

证明.

任取状态 σ , 如果 $\sigma \models B$, 则有

$$\langle \text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma \rangle \rightarrow \langle S_1, \sigma \rangle,$$

二者终止于相同状态或同时发散; 如果 $\sigma \not\models B$, 则有

$$\langle \text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi}, \sigma \rangle \rightarrow \langle S_2, \sigma \rangle,$$

二者终止于相同状态或同时发散. □

定义 1.2 (while 的迭代程序与算子)

给定 $B : \text{Bool}$ 和程序 S .

1. 递归定义程序 A^k ($k \in \mathbb{N}$):

$$A^k = \begin{cases} \text{while true do skip od}, & k = 0, \\ \text{if } B \text{ then } S; A^{k-1} \text{ else skip fi}, & k > 0. \end{cases}$$

2. 定义算子 $\Phi : [\Sigma \rightarrow \Sigma] \rightarrow [\Sigma \rightarrow \Sigma]$: 对任意部分函数 $f : \Sigma \rightarrow \Sigma$,

$$\Phi(f) = f \circ \llbracket S \rrbracket \upharpoonright_B \cup \text{id}_{\neg B},$$

易证它单调且保上确界, 从而 $\bigcup_{k \in \mathbb{N}} \Phi^k(\emptyset) = \text{fix } \Phi$.

定理 1.4 (while 语句的指称语义)

对任意的 $B : \text{Bool}$ 和程序 S , 有

$$\llbracket \text{while } B \text{ do } S \text{ od} \rrbracket = \bigcup_{k \in \mathbb{N}} \llbracket A^k \rrbracket = \bigcup_{k \in \mathbb{N}} \Phi^k(\emptyset) = \text{fix } \Phi.$$

证明.

记 $W = \text{while } B \text{ do } S \text{ od}$.

1. 对任意的 $\tau = \llbracket W \rrbracket(\sigma)$, 记该计算中 W 出现次数为 $n \geq 1$, 对 n 归纳证明 $\tau = \llbracket A^n \rrbracket(\sigma)$.

首先, 如果计算中 W 只出现 1 次, 则一定有 $\sigma \not\models B$, 从而 $\tau = \sigma = \llbracket A^1 \rrbracket(\sigma)$;

其次假设命题对 n 成立, 并且计算中 W 出现了 $n+1$ 次, 则一定有 $\sigma \models B$. 此时 $\langle W, \sigma \rangle \rightarrow \langle S; W, \sigma \rangle$ 终止, 从而 $\langle S, \sigma \rangle$ 一定终止, 设其终止于 σ' , 则

$$\langle W, \sigma \rangle \rightarrow \langle S; W, \sigma \rangle \rightarrow^* \langle W, \sigma' \rangle, \quad \langle A^{n+1}, \sigma \rangle \rightarrow \langle S; A^n, \sigma \rangle \rightarrow^* \langle A^n, \sigma' \rangle$$

此时 $\tau = \llbracket W \rrbracket(\sigma')$, 易证 $\langle W, \sigma' \rangle$ 的计算中 W 出现 n 次, 依假设有 $\tau = \llbracket A^n \rrbracket(\sigma') = \llbracket A^{n+1} \rrbracket(\sigma)$.

归纳成立, 从而 $\llbracket W \rrbracket \subset \bigcup_{k \in \mathbb{N}} \llbracket A^k \rrbracket$.

2. 对 k 归纳证明 $\llbracket A^k \rrbracket \subset \llbracket W \rrbracket$. 显然 $k = 0$ 时 $\llbracket A^0 \rrbracket = \emptyset$; 假设命题对 k 成立, 任取 $\tau = \llbracket A^{k+1} \rrbracket(\sigma)$.

如果 $\sigma \models B$, 此时 $\langle A^{k+1}, \sigma \rangle \rightarrow \langle S; A^k, \sigma \rangle$ 终止于 τ , 从而 $\langle S, \sigma \rangle$ 一定终止, 设其终止于 σ' , 则

$$\langle A^{k+1}, \sigma \rangle \rightarrow \langle S; A^k, \sigma \rangle \rightarrow^* \langle A^k, \sigma' \rangle, \quad \langle W, \sigma \rangle \rightarrow \langle S; W, \sigma \rangle \rightarrow^* \langle W, \sigma' \rangle,$$

从而依假设有 $\tau = \llbracket A^k \rrbracket(\sigma') = \llbracket W \rrbracket(\sigma') = \llbracket W \rrbracket(\sigma)$.

如果 $\sigma \not\models B$, 则 $\tau = \sigma = \llbracket W \rrbracket(\sigma)$. 所以始终有 $\llbracket A^{k+1} \rrbracket \subset \llbracket W \rrbracket$.

归纳成立, 从而 $\bigcup_{k \in \mathbb{N}} \llbracket A^k \rrbracket \subset \llbracket W \rrbracket$. 综上可得二者相等.

3. 对 k 归纳证明 $\llbracket A^k \rrbracket = \Phi^k(\emptyset)$. 显然 $\llbracket A^0 \rrbracket = \emptyset = \Phi^0(\emptyset)$, 假设命题对 k 成立, 则

$$\begin{aligned} \llbracket A^{k+1} \rrbracket &= \llbracket \text{if } B \text{ then } S; A^k \text{ else skip fi} \rrbracket \\ &= \llbracket S; A^k \rrbracket \upharpoonright_B \cup \llbracket \text{skip} \rrbracket \upharpoonright_{\neg B} \\ &= \llbracket A^k \rrbracket \circ \llbracket S \rrbracket \upharpoonright_B \cup \text{id}_{\neg B} \\ &= \Phi(\llbracket A^k \rrbracket) = \Phi^{k+1}(\emptyset), \end{aligned}$$

归纳成立, 从而 $\bigcup_{k \in \mathbb{N}} \llbracket A^k \rrbracket = \bigcup_{k \in \mathbb{N}} \Phi^k(\emptyset)$. □

2 指称语义的其他形式

定义 2.1 (全函数形式)

程序 S 的全函数形式的指称语义定义为

$$\llbracket S \rrbracket_{\text{tot}} : \Sigma \cup \{\perp\} \rightarrow \Sigma \cup \{\perp\}, \quad \sigma \mapsto \begin{cases} \llbracket S \rrbracket(\sigma), & \sigma \in \Sigma \text{ 且 } \llbracket S \rrbracket(\sigma) \text{ 有定义,} \\ \perp, & \text{其它.} \end{cases}$$

定理 2.1

1. 对任意的程序 S_1, S_2 , $\llbracket S_1; S_2 \rrbracket_{\text{tot}} = \llbracket S_2 \rrbracket_{\text{tot}} \circ \llbracket S_1 \rrbracket_{\text{tot}}$,
2. 对任意的 $B : \text{Bool}$ 和程序 S_1, S_2 , 有

$$\llbracket \text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi} \rrbracket_{\text{tot}} = \llbracket S_1 \rrbracket_{\text{tot}} \upharpoonright_B \cup \llbracket S_2 \rrbracket_{\text{tot}} \upharpoonright_{\neg B} \cup \{(\perp, \perp)\}.$$

定义 2.2 (集合值形式)

程序 S 的集合值形式的部分正确性语义定义为

$$\mathcal{M}[\llbracket S \rrbracket] : \mathcal{P}(\Sigma) \rightarrow \mathcal{P}(\Sigma), \quad X \mapsto \{\tau \in \Sigma \mid \exists \sigma \in X : \langle S, \sigma \rangle \text{ 终止于 } \tau\},$$

其完全正确性语义定义为

$$\mathcal{M}_{\text{tot}}[\llbracket S \rrbracket] : \mathcal{P}(\Sigma \cup \{\perp\}) \rightarrow \mathcal{P}(\Sigma \cup \{\perp\}), \quad X \mapsto \mathcal{M}[\llbracket S \rrbracket](X \cap \Sigma) \cup \{\perp \mid \perp \in X \vee \exists \sigma \in X : \langle S, \sigma \rangle \text{ 发散}\}.$$

定理 2.2

1. 对任意的程序 S 和 $X \subset \Sigma$, $\mathcal{M}[\llbracket S \rrbracket](X) = \llbracket S \rrbracket[X]$,
2. 对任意的程序 S 和 $X \subset \Sigma \cup \{\perp\}$, $\mathcal{M}_{\text{tot}}[\llbracket S \rrbracket](X) = \llbracket S \rrbracket_{\text{tot}}[X]$.

定理 2.3

下文 $\mathcal{N}$ 指代 $\mathcal{M}$ 或 $\mathcal{M}_{\text{tot}}$, 其中 $\mathcal{M}$ 自动忽略输入的 $\perp$.

1. 对任意的程序 S_1, S_2 , 有 $\mathcal{N}[\llbracket S_1; S_2 \rrbracket](X) = \mathcal{N}[\llbracket S_2 \rrbracket] \circ \mathcal{N}[\llbracket S_1 \rrbracket]$,
2. 对任意的 $B : \text{Bool}$, 程序 S_1, S_2 和 $X \subset \Sigma \cup \{\perp\}$, 有 $\mathcal{N}[\llbracket \text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi} \rrbracket](X) = \mathcal{N}[\llbracket S_1 \rrbracket](X \cap B) \cup \mathcal{N}[\llbracket S_2 \rrbracket](X \cap \neg B) \cup \{\perp \mid \perp \in X \wedge \mathcal{N} = \mathcal{M}_{\text{tot}}\}$.
3. 对任意的 $B : \text{Bool}$ 和程序 S 和 $X \subset \Sigma \cup \{\perp\}$, 有

$$\mathcal{M}[\llbracket \text{while } B \text{ do } S \text{ od} \rrbracket](X) = \bigcup_k \mathcal{M}[A^k](X).$$